

Y8960 TECH DEMO

Cとz88dkで作る、音源デモの仕組み
音源I/O・曲データ・ADPCM・描画・検証
0.5.0 / 2026-09-13



Y8960 TECH DEMOの楽しみ方

音を聴く

Y8960は、HRA!さんが設計したMSX用の拡張音源カートリッジです。

このデモはY8960対応openMSXで演奏します。
openMSXはMSXの動作をパソコン上で再現するソフトです。

約32秒の曲を1～4キーで聴き比べます。

画面を見る

青系の背景に白い文字と影を配置し、1re1さんのMSX 8x8 fontを使用しています。

8本のバーで各パートの発音を確認できます。
バーは音量の測定値ではありません。

起動方法と動作環境

Windows側

setup-y8960.bat：検証済みエミュレーターZIPを展開し、起動試験を実施。

launch-y8960-tech-demo.bat：専用設定でROMを実行。通常利用にz88dkやPythonは不要です。

MSX側

C-BIOS MSX2+、Z80、60 Hz。
通常のV9958で画面を描画します。

カートリッジA：16 KiBのデモROM。
カートリッジB：HRA_Y8960。

V9968・商用BIOS・実機BIOSは使いません。

Y8960とは何か

MSXへ音の表現を追加

MSXは1980年代に登場したパソコンの規格です。拡張カートリッジを接続し、機能を追加できます。

Y8960は複数の音源を組み合わせた拡張音源です。MSXから命令を送り、音程や音色、音量を指定します。

このデモで使う環境

Y8960対応openMSXとC-BIOSを使用します。

C-BIOSは、MSXの基本機能を提供するフリーの代替BIOSです。

デモの起動にMSX実機や実機付属のBIOS ROMは必要ありません。

音源ごとの音の作り方

SSG：簡単な波形で音を作る

矩形波やノイズを使います。輪郭のはっきりした電子音が特徴です。

FM：波形を変化させて音色を作る

波形同士を組み合わせ、金属的な音や楽器を思わせる音を作ります。OPLLとOPL2はFM音源です。

ADPCM：音声データを再生する

音の波形を圧縮して保存し、再生します。このデモでは打楽器の音に使用します。

楽器を演奏する人にたとえると、音源は音を出す担当、プログラムは音程とタイミングを指示する役割です。

PSGとSSGの関係

項目	PSG	SSG
呼び名	Programmable Sound Generator	Software-controlled Sound Generator
代表的なIC	GI AY-3-8910	Yamaha YM2149
音の作り方	矩形波・ノイズ・音量変化	PSGと同系統の仕組み
関係	MSXの標準音源の呼び名	ヤマハの呼称。音の特性には差
Y8960での位置づけ	PSG系の機能を提供	作者はヤマハSSGを模したコアを採用

SSGはPSGと大きく異なる方式ではありません。実装による音の差はあります。

OPLLとOPL2の違い

項目	OPLL / YM2413	OPL2 / YM3812
身近な製品	FM-PAC / MSX-MUSIC	初期Sound Blaster / AdLib
音の仕組み	2オペレーターFM	2オペレーターFM
同時発音	9音、または6音 + 打楽器5種	9音、または6音 + 打楽器5種
音色	内蔵15種類 + 自作1種類	各チャンネルで音色を設定
自作音色の扱い	設定1組を全チャンネルで共有	チャンネルごとに異なる設定
使い方	音色番号を選び手軽に演奏	パラメーターで音色を作り込む

標準チップの比較です。Y8960独自の拡張や、Sound BlasterのPCM再生は別機能です。

なぜ複数の音源を組み合わせるのか

役割を分担して合奏する

旋律、和音、低音、打楽器を別々に鳴らすと、曲に厚みやリズムが生まれます。

音の作り方が違う音源を重ねると、単独の音源とは違う響きになります。

このデモでは少ない音から順に重ね、その違いを体験できます。

組み合わせる例

SSG：旋律の輪郭

FM：和音・低音の厚み

ADPCM：打楽器のリズム

3つの役割を重ねて、1つの曲へ

フォークの実装とデモの利用を分ける

音源の役割

SSG：波形を使う素朴な電子音。

OPLL：FM合成による旋律と和音。

OPL2：FM合成による低音。

ADPCM：圧縮した音声の再生。

同じ旋律に音を重ね、それぞれの響きを聴き比べます。

実装と利用範囲

このデモはSSG、OPLL、OPL2、ADPCMを使います。

DCSGやSCC、タイマー割り込みによる演奏は含みません。

フォークの全機能・全チャンネルを試すものではありません。

4セクションで音の役割を分ける

01 SEED → 02 MIRROR

SEED：SSGの単音旋律。音程と輪郭を聴きます。

MIRROR：もう一方のSSGで4ステップ前の旋律を再生し、左右の掛け合いとSSG低音を加えます。

エコーは過去の音声の録音再生ではありません。

03 ALLOY → 04 PULSE

ALLOY：2系統のOPLLで旋律を重ね、和音を加えます。

PULSE：OPL2低音と、キック／スネアのADPCM発音を追加。

各セクション32ステップ、約8秒。128ステップで最初へ戻ります。

楽譜の代わりに、音番号を並べる

旋律・和声の素材

melody：64個の音番号。0は休符。

bass_notes：38, 34, 43, 45。

旋律は16ステップ×4フレーズ。

和声の土台はD minor／B♭／G minor／Aです。

64ステップ分の素材を2回使用し、128ステップの演奏を作ります。

編集時に確認すること

音番号nの基準周波数：

$$f = 440 \times 2^{((n-69)/12)}$$

音程表は0～95を生成します。n-12、b+31などの参照も範囲内か確認します。

音色・音量・パンの変更は曲データとは別にplay()／audio_init()で指定します。

演奏のタイミングを描画から分ける

時間を測る仕組み：JIFFY（ジフィー）

MSXの基本ソフト（BIOS）が、画面の更新周期に合わせて1ずつ増やす数値です。

増えた数から、経過時間を求められます。

1秒に約60回、50Hz設定では50回増えます。

このデモは約60Hzを前提とし、15回分（約0.25秒）で曲を1ステップ進めます。

曲は128ステップ、約32秒で一巡します。

演奏を先に、画面更新は少しずつ

前回から増えたJIFFYの値を読み、次の音を鳴らす時刻かどうかを先に判定します。描画の回数で曲を進めないため、画面処理の重さによるテンポのずれを抑えます。

音源への命令を送り、その後で見出しやバーなどを1件だけ更新します。次の発音直前や画面チップの処理中は、描画を後回しにします。

音の処理もCPUを使うため、遅れを完全になくす仕組みではありません。

ROMを保ったまま音源I/Oを有効化

スロットの前提

C-BIOS MSX2+のカートリッジBは基本スロット2です。

BIOS WRTSLT (0x0014) で、スロット2の0x7FF6へ0x03、0x7FFFへ0x9Fを書きます。

実行中のROMがあるスロットを、Cコード自身で切り替えません。

有効化後の確認

0x7FF6：OPLLの2系統を有効化。

0x7FFF：OPL2、DCSG、SSGS、タイマーを有効化する値。

デモではDCSGを鳴らしません。

B0へ0x55を書き、読み戻してI/O経路を確認します。全音源の動作証明にはなりません。

音源ごとにアドレスとデータを送る

使用ポート（16進数）

SSGS : A0 / A1

OPLL 0 : 7C / 7D

OPLL 1 : 7A / 7B

OPL2 0 : C0 / C1

OPL2 1 : C2 / C3

各組はレジスタ番号／値の順です。

Cプログラムから音源へ命令を送る

ssg()、fm()、opl()は、それぞれの音源に設定を書き込む関数です。

まず変更する設定の番号、次に値を送ります。
この2回の書き込みの間に別の処理が入らないよう、一時的に割り込みを禁止します。

実機で必要な待ち時間や別CPUでの動作は、
このエミュレーター試験だけでは保証しません。

SSGS：音程・音量・左右の掛け合い

音程とチャンネル

周期 $P \approx 1,789,772.5 / (16f)$ 。
12bit周期を2レジスタへ書きます。

unit 0はレジスタ群0x00、unit 1は0x20。音量は8～10、パンは0x10～0x12を基準にします。

音番号0では音量を0にします。

ステレオの配置

主旋律のパン設定は2、エコーは13です。

SSG低音は4で、主旋律側に寄せています。設定8のチャンネルは未使用です。

4ステップ前の旋律をもう一方の音源で演奏し、掛け合いを作ります。

OPLL：2系統で旋律と和音を重ねる

音程データ

f-numberとblockを生成時に計算し、表から読みます。

$$F \approx f \times 2^{(19 - \text{block})} / 49,716$$

キーOFF → 音色・音量 → 音程 → キーONの順に設定します。

重ね方

系統0：プリセット3、主旋律。

系統1：プリセット4、1オクターブ下。

4ステップごとに、両系統の別チャンネルへ和声音を送ります。

両方を鳴らすことと、両方を左右へ独立パンすることは同じではありません。

OPL2：FMベースの音色を作る

2オペレーターの設定

0x20/0x23：倍率など。

0x40/0x43：出力レベル。

0x60/0x63：アタック／ディケイ。

0x80/0x83：サステイン／リリース。

0xC0：接続・フィードバック。

設定はaudio_init()に集約しています。

このデモでの役割

PULSEでは、2ステップごとに低音を鳴らします。

OPL2は低音専用の音源ではありません。このデモでは系統0を低音、系統1を打楽器の再生に使用します。

音程表は曲の生成時に計算します。

ADPCM素材はPythonで合成する

キックとスネア

キック：下降する正弦波と短いノイズ。
スネア：ノイズと185 Hzの正弦波。

指数減衰で余韻を作ります。乱数seedは8960に固定し、再生成可能にします。

サンプルのサイズ

各音：11,025 Hzで4,096 PCM値。
長さは約0.372秒です。

4bit ADPCMに変換すると各2,048バイト、2音で4,096バイト。

商用音源や既存曲の録音を収録する構成ではありません。

ADPCMの符号化と共有RAM

符号化の流れ

前回復号値との差から符号と大きさを決定し、4bitに量子化します。

復号予測値とステップ幅を更新。
ステップ幅は127～24,576に制限。

2つの4bit値を、上位→下位の順で1バイトへ詰めます。

RAMの転送と検査

4,096バイトを音源の共有メモリへ送ります。
前半がキック、後半がスネアです。

テストはメモリを読み戻し、生成したデータと全バイトを照合します。

単独で録音した打楽器の音量も検査します。

ADPCMを2系統から発音する

開始・終了アドレス

開始／終了レジスタの単位は4バイトです。

キック：開始0、終了511。

スネア：開始512、終了1,023。

2個目は別RAMではなく、共有RAMの後半を指します。

再生速度と音量

$\Delta N = 0x38C5$ 。

$\approx 49,716 \times \Delta N / 65,536$

$\approx 11,025$ サンプル／秒。

レベル175、制御0xA0で再生。

4ステップごとにキック／スネアを交互に発音します。

SCREEN 5で文字に3色を使う

画面とキャッシュ

SCREEN 5、256×212、16色。
ページ0が表示領域です。

背景4：RGB(1,2,3)。
文字15：RGB(7,7,7)。
影1：RGB(0,1,1)。

見出しと状態表示を、起動時に非表示ページ1
へ描きます。

白い文字と、右下1ピクセルの影



CPUが文字と影を横1行ずつ作り、画面用メモリへ送ります。Z80の連続出力命令（OTIR）を使い、文字の高さと影を合わせた9行を転送します。

演奏中は、起動時に準備した文字画像を画面チップがコピーします。

描画コマンドと割り込みの整合

描画を小分けにする

1回の処理では、見出し・状態表示・バー1本などを描きます。

次の音を鳴らす直前や、画面チップが処理中なら、描画を後回しにします。

音を優先し、表示の遅れを許容する設計です。

画面チップへの命令

HMMM：画面用メモリ内の画像をコピーします。完了を待たず、CPUは次の処理へ進みます。

HMMV：進行バーなどの長方形を塗ります。

命令の完了状態を確認してから、次の描画を開始します。

描画を小分けにする具体的な順序

更新するものを1件選ぶ

1. セクションが変わったら見出しをコピー。
2. 再生・消音・一時停止状態が変わったら更新。
3. バー1～8、進行バーを順番に更新。

バーは16×16スプライトを上下2枚で表示。
8本で16枚、同じ水平走査線では8枚です。
高さ別の画像を起動時に準備し、更新時は
上下のパターン番号2バイトを書き換えます。

実画面のバー8本を拡大



1本は16×16ピクセルの画像を上下に2枚配置。
合計16枚ですが、横に並ぶのは8枚です。

8本と進行バーの更新には最低9回必要で、
約60Hzなら一巡は約150msが目安です。
次の発音直前や画面チップの処理中は待ち、
バーの減衰は各バーの更新時に進めます。

ROM・RAM・VRAMを混同しない

ROMとCPU RAM

ROM：16 KiB、Normal4000。

BSS：0xC100より手前で終了。

0xC100～0xC107：テスト用状態。

0xC200～0xC27F：128バイトの描画バッファ。

ヒープは使いません。

VRAMと音源RAM

表示：ページ0、Y=0～211。

スプライト：色0x7400、属性0x7600、
高さパターン0x7800～0x7A1F。

見出し：ページ1、Y=256～383。

状態：Y=384、400、416の各9行。

スプライトと文字キャッシュは重なりません。

ADPCM共有RAMは別空間です。

入力と停止は用途を分ける

1～4／Space／M

1～4：対象セクションの先頭へ。

Space：時間進行を止めて消音。再開時に現在の音を送ります。

M：時間を進めたまま消音。

キーマトリクスを読み、押し始めのエッジで切り替えます。

R／Escと既知の制限

R：ステップと時間を0へ戻します。

Esc：使用音源を止めて停止表示。

ウィンドウは自動では閉じません。

描画中の短いキー入力は遅れたり、取りこぼす可能性があります。

現行の検証はエミュレーター上です。

セットアップと配布ファイル

利用するBATは2本

setup-y8960.bat

必要なファイルを検査・配置します。

launch-y8960-tech-demo.bat

準備済みの環境でデモを起動します。

通常利用に開発ツールは不要です。

GitHub Releasesから取得

y8960-tech-demo-0.5.0.zipを展開します。

setupは固定したエミュレーターを取得し、SHA-256と起動結果を検査します。

同じReleaseに対応ソースも掲載します。
第三者のライセンス表示は同梱文書に記載しています。

再ビルドと、公開対象の検査

開発用のビルド

z88dkとPython 3を用意します。

音楽・画像データを生成し、CソースからROMを作成します。

ビルド時に、音番号の範囲・ROMサイズ・メモリの境界を検査します。

テストの入口

tests/run-all.ps1

起動、操作、音声、画面、ADPCM、発音時刻を一括検査します。

配布ZIPは公開一覧に従って作成し、収録ファイルを照合します。

利用条件の確認は別途必要です。

自動テストで確認していること

発音時刻の結果と許容値

消音→発音：8.507ms／許容10ms。
境界の誤差：8.063ms／許容16.688ms。
通常間隔の誤差：23.002ms。
累積ずれ：23.003ms。

後者2つの許容値は33.376msです。1フレーム程度の遅れを許容する検査で、厳密な同期の保証ではありません。

現在のROMで再検査

70秒、280音、8境界でPASS。

音源命令と録音の開始時刻の差は0.291msで、許容5ms以内です。

録音の音量低下、操作、文字とスプライトも検査します。

PC側の音声バッファ切れは対象外です。

音の途切れを検出する方法

検出器の負例テスト

1音の欠落、100msの遅延、50msの無音を検出します。

弱い残響がある音量低下や、録音時刻の25msのずれも検出します。

異なるROMや音楽データの記録は解析前に拒否します。

検査できる範囲

音源への命令時刻と録音を調べます。PC側の音声バッファの途切れや、すべての雑音を判定するものではありません。

実機、別CPU、50Hz環境は、個別の検証が必要です。

出典・謝辞と、再現対象

実装の一次資料

madscient/openMSX_Y8960

コミット：78469c4

本リポジトリのCソース、音楽生成ツール、テストを参照できます。

詳しい参照先と再実行方法は開発ドキュメントに記載しています。

謝辞と利用条件

HRA!さん：Y8960の設計。

buppu3さん：初期エミュレーター。

madscientさん：拡張フォーク。

1re1さん：MSX 8x8 font。

第三者の素材は各自の条件に従います。

確認日：2026-09-13。